

개발 경로 한눈에

01 / START HERE · 현재 업무: 메모리 read/write 기반. 영상은 비교 사례.

현재 업무의 읽는 순서

2~7쪽 / 07 원문	ST 의존 없는 memory read/write + UART 모델
8~11쪽	handle·함수 포인터·데이터시트·bit field
12~15쪽	영상의 HAL/LL 비교 + C 가변 인자
16~21쪽	decode·timeout·ISR·C/C++ 문법

처음 2~3시간

0~30분	주소 공간·명령 폭·clock/reset 문서 확인
30~60분	안전한 register read 한 번 구현
60~100분	설정 write·상태 확인·한 byte 송신
100~140분	timeout·부분 실패·재초기화 계약
140~180분	파형·실제 baud·상대 수신으로 확인

전체 파일 세트

- examples/memory_io/: 현재 업무용, ST HAL 의존 없음.
- examples/uart_template.*: 이전 STM32 비교 학습용.
- raw_uart는 실제 부품 map이 없는 교육용 모델.
- 영상 정리·기존 분석은 그대로 보존했다.

스니펫 표기

C17 / C++17	헤더·의존 파일·전제를 확인하고 사용
HAL	STM32 비교 사례. 현재 업무에 필수 아님.
구조 예시	전체 코드의 발췌 또는 문맥을 채울 조각

영상에서 다시 볼 위치

01:42	센서 회로: 전원·통신 선택·주소
04:16	CubeIDE: 클록·GPIO·I2C 준비
06:40	.h/.c 책임 분리
07:55	레지스터 상수·구조체·API
16:11	HAL을 감싼 read/write 함수
18:41	초기화·ID 확인
21:38	설정 요구 → 레지스터 비트
26:18	온도 12비트 + 변환식
30:21	가속도 20비트 + 부호·단위
34:10	DRDY·반복 읽기·실물 확인

영상의 설정을 읽는 법

장치	STM32F405 + ADXL355 + I2C
FILTER 0x28	0x05: HPF 비활성, ODR 125 Hz, LPF 31.25 Hz
POWER_CTL 0x2D	영상은 0x00: 측정·온도·DRDY 활성화
개발 포인트	숫자를 외우지 말고 필드와 요구사항을 연결

검증 범위와 원문 관계

00~07, 목차, 출처 문서의 핵심을 압축했다. 영상의 기존 설명과 현재 업무용 확장은 구분한다.

- 영상 직접 내용과 이후 확장 설계는 구분한다.
- 현행 온도 기준은 1885 LSB@25°C. 영상의 1852와 구분한다.
- 과거 영상과 현재 공개 master 코드는 완전히 같지 않다.
- 실물 시험 미수행. C 시험 범위는 각 예제 README에 표시.

내 업무: 주소 + 메모리 명령

02 / RAW ACCESS · ST HAL/LL 없이 직접 작성하는 경로

가장 먼저 확인할 4가지

주소 공간	명령 주소인가, CPU MMIO 주소인가?
접근 단위	offset은 byte? word? data 접근 폭은?
성공의 의미	명령 수락, bus 전달, 장치 완료 중 무엇인가?
읽기 부작용	단순 상태인가, RC-FIFO처럼 읽으면 바뀌는가?

내가 만드는 계층

- 응용 → RawUart 같은 내 API를 호출한다.
- 드라이버 → offset·mask·순서·상태를 관리한다.
- 접근 계층 → 주어진 read/write 명령을 연결한다.
- 보드 → 전원·clock·reset·pinmux를 준비한다.
- MemIo는 이 자료의 직접 정의 타입이다. ST 라이브러리가 아니다.

콜백 계약의 모양

S40 구조 예시

40_memory_api.h

```
typedef struct {
    void *ctx;
    MemStatus (*read32)(
        void *, MemAddr, uint32_t *,
        uint32_t budget_ms);
    MemStatus (*write32)(
        void *, MemAddr, uint32_t,
        uint32_t budget_ms);
    MemStatus (*sync)(void *, uint32_t);
    uint32_t (*now_ms)(void *);
} MemIo;
```

mem_io.h 발췌. MemAddr=uint64_t, MemStatus도 같은 전체 헤더에 정의. 중복 정의하지 않는다.

명령 주소는 포인터가 아닐 수 있다

probe·원격 command-PCIe 접근 주소를 호스트 C 포인터로 바로 바꾸지 않는다. 받은 API의 주소 공간을 유지한다. CPU에서 유효한 장치 매핑을 받은 경우에만 포인터 접근을 검토한다.

backend에 요구하는 보장

read32	정렬된 단일 32-bit 접근. 성공 시 CPU 순서 정수.
write32	단일 32-bit 쓰기. 실패해도 도달했을 수 있음.
sync	플랫폼이 요구하는 선행 쓰기 순서·전달 보장.
now_ms	polling 중에도 진행. unsigned wrap 허용.
timeout	각 callback에 남은 시간 전달. 반환 시간 상한 필요.
수명	동기 함수. 버퍼 미보관. ctx는 사용 중 생존.

무엇을 복사하는가

- examples/memory_io/mem_io.h + .c
- examples/memory_io/raw_uart.h + .c
- port_binding.example.c: 실제 명령 연결 자리
- test_memory_io.c: 실제 주소 없는 RAM 모델 시험
- 07 문서에서 포팅 순서와 계약을 먼저 확인한다.

실제 register map은 미지정

칩·주소·명령 원형이 아직 없으므로 실제 address/mask/ baud 값을 임의로 넣지 않았다. UART 모델의 순서가 실제 칩과 다르면 숫자뿐 아니라 알고리즘도 수정한다.

주소·폭·정렬·CPU 접근

03 / ADDRESSING · 데이터 비트 수와 bus 접근 폭은 별개의 조건

범위를 검사한 register read

S41 C17

41_read_at32.c

```
MemStatus read_at32(
    const MemIo *io, MemAddr base,
    uint32_t span, uint32_t offset,
    uint32_t *out, uint32_t budget)
{
    MemAddr address;
    if (!MemAddr_At32(base, span,
                      offset, &address)) {
        return MEM_EARG;
    }
    return Mem_Read32(io, address,
                      out, budget);
}
```

전체 mem_io.h/c와 사용. base-offset은 byte 주소. 범위 검사가 register 부작용을 검증하지는 않음.

포인터 덧셈의 단위

정수 주소 + 0x10	byte-address 계약이면 16 byte 이동
uint32_t *p + 0x10	보통 64 byte 이동. sizeof 원소만큼 배율 적용.
span 검사	시작점만이 아니라 마지막 접근 byte까지 포함
주소 overflow	base + offset 계산 전에 표현 범위를 검사

너비를 바꾸면 별도 accessor

8-bit 값이라고 read32를 read8로 임의 교체하지 않는다. 반대로 8-bit register에 write32로 쓰면 이웃 register를 건드릴 수 있다. 64-bit 값을 32-bit 두 번으로 읽는 순서·일관성도 장치별 규칙이다.

조건부 CPU MMIO 조각

S42 구조 예시

42_cpu_mmio.h

```
#include <stdint.h>

static inline uint32_t cpu_load32(
    uintptr_t mapped_addr)
{
    return *(volatile const uint32_t *)
        mapped_addr;
}

static inline void cpu_store32(
    uintptr_t mapped_addr, uint32_t v)
{
    *(volatile uint32_t *)mapped_addr = v;
}
```

CPU 접근 가능한 올바른 장치 매핑·32-bit 정렬·폭·toolchain 계약 필요. command 주소에 적용 금지.

이 조각이 해주지 않는 것

- 물리 주소를 접근 가능한 가상 주소로 매핑하지 않는다.
- bus fault를 MemStatus 오류로 자동 변환하지 않는다.
- endian·memory attribute·barrier·cache를 해결하지 않는다.
- volatile은 lock이나 원자적 RMW가 아니다.

overlay struct는 선택 사항

register struct를 쓰려면 offsetof·padding·정렬·접근 폭을 실제 map과 대조한다. packed나 bit-field를 붙여도 외부 layout과 atomicity가 보장되지 않는다. 명령 API는 명시적인 offset만으로 충분하다.

read/write에 register 의미 부여

04 / REGISTER POLICY · 같은 주소 폭이어도 읽고 쓰는 규칙은 다르다.

접근 정책 빠른 선택

plain RW	전체 안전한 값 쓰기 또는 보호된 RMW
RO	읽기만 허용. 부작용 유무는 별도 확인.
WO	정해진 값 write. readback 가정 금지.
W1C	지울 bit만 1. 나머지 0 쓰기 안전 조건.
W0C	0으로 지움. reserved까지 ~mask 금지.
RC / FIFO	읽기가 상태를 바꿈. dump-logging도 접근.
혼합 register	필드별 의미에 맞는 장치 전용 write

plain RW 필드 교체

S43 구조 예시

43_rw_field.c

```
uint32_t old, next;
st = Mem_Read32(&io, addr, &old, left);
if (st != MEM_OK) return st;
if (!Mem_FieldReplace32(old, mask,
                        bits, &next)) {
    return MEM_EARG;
}
/* Recompute remaining budget here. */
st = Mem_Write32(&io, addr, next, left);
```

전체 RMW 동안 소유권 보호. bits는 이미 shift된 값. plain RW-reserved 정책 확인. sync 필요성 별도.

전용 W1C acknowledge

S44 구조 예시

44_ack_w1c.c

```
st = Mem_Write32(&io, ack_addr,
                handled_flags, left);
```

전용 W1C이고 나머지 0 쓰기가 안전한 경우만. read한 전체 값에 OR하지 않는다.

교육용 UART의 map/config

S45 구조 예시

45_uart_map.h

```
typedef struct {
    MemAddr base;
    uint32_t span_bytes;
    uint32_t status_off, txdata_off;
    uint32_t ctrl_off, baud_off;
    uint32_t tx_ready_mask;
    uint32_t tx_idle_mask;
    uint32_t ctrl_disabled;
    uint32_t ctrl_enabled;
    uint32_t baud_value;
} RawUartConfig;
```

raw_uart.h 발췌. 실제 address-mask-divider를 제공하지 않는다. 전체 헤더와 중복 정의 금지.

모델에만 적용되는 가정

- 서로 다른 4개 register, 모두 정렬된 32-bit 접근.
- STATUS는 반복 읽어도 안전. READY/IDLE은 active-high.
- TXDATA는 bits[7:0] FIFO push. 상위 비트 0 쓰기 안전.
- CTRL/BAUD는 전체 값 쓰기가 안전한 plain RW.
- 실제 map에 bank-DLAB-unlock이 있으면 흐름도 바꾼다.

셋업 값을 계산할 때

ctrl_disabled/enabled는 reserved 규칙을 만족하는 전체 값이다. baud_value는 encode된 값이며 baud rate 자체가 아니다. 입력 clock·prescaler·oversampling·분수 divider·오차를 실제 식으로 계산한다.

읽는 것도 동작이다

RC/FIFO에 대한 반복 readback은 이벤트-데이터를 소비할 수 있다. 디버거의 자동 register 갱신도 끈다. 쓰기 확인용 읽기는 문서상 안전한 register에 한정한다.

내 UART handle과 초기화

05 / RAW UART INIT · 타입을 직접 정의하고 알려진 상태에서 시작

타입이 있는 API

S46 C17

46_raw_uart_api.h

```
MemStatus RawUart_Init(
    RawUart *h, const MemIo *io,
    const RawUartConfig *cfg,
    uint32_t timeout_ms);

MemStatus RawUart_Write(
    RawUart *h, const uint8_t *src,
    size_t n, size_t *sent,
    uint32_t timeout_ms);

MemStatus RawUart_Flush(
    RawUart *h, uint32_t timeout_ms);
```

전체 raw_uart.h/c 사용. RawUart는 내 객체이며 ST handle이 아님.

handle과 config의 소유권

RawUart h={0}	OFF 상태로 생성. 사용 기간 동안 생존.
&h	원래 객체의 주소. 함수는 h->state 등을 갱신.
config	init이 값 복사. 호출 뒤 cfg 원본 수명 불 필요.
io 함수 포인터	handle에 복사. callback의 ctx 객체는 빌림.
동시 호출	이 버전은 외부에서 직렬화. ISR용 API 아님.

uint8_t *handle이 아닌 이유

handle은 바이트 배열이 아니라 주소·설정·상태 객체다. uint8는 표준 이름이 아니며 uint8_t는 데이터 byte에 사용한다. 가변 인자 ... 대신 config로 옵션 타입을 명시한다.

실제 호출 모양

S47 구조 예시

47_raw_uart_use.c

```
RawUart uart = {0};
MemStatus st = RawUart_Init(
    &uart, &board_io, &board_cfg, 100U);
if (st == MEM_OK) {
    const uint8_t msg[] = {'O', 'K'};
    size_t sent = 0U;
    st = RawUart_Write(&uart, msg,
        sizeof msg, &sent, 100U);
    if (st == MEM_OK) {
        st = RawUart_Flush(&uart, 100U);
    }
    /* Handle st and partial sent. */
}
```

board_io·board_cfg는 실제 명령과 map에서 구성. 전원·clock·reset·pinmux 선행.

Init의 모델 순서

1. 검증	주소·폭·범위·mask·인자 확인
2. FAULT	설정 복사. 아직 성공 상태로 표시하지 않음.
3. 설정	disable → sync → baud → sync → enable → sync
4. READY	전체가 성공한 마지막에 사용 가능으로 변경

실제 칩에 추가할 것

reset/ready 대기, FIFO reset, unlock, bank/alias, baud 공식은 실제 데이터시트로 채운다. Init 실패는 일부 쓰기 적용 상태일 수 있다. 코드가 자동으로 원상복구했다고 가정하지 않는다.

FIFO·timeout·부분 실패

06 / RAW UART TX · 쓰기 성공과 마지막 stop bit 완료를 구분

TX 알고리즘

S48 구조 예시

48_tx_sequence.txt

```
start = now_ms();
for each byte:
    left = timeout - elapsed(start);
    poll STATUS until TX_READY;
    write32(TXDATA, byte, left);
    increment accepted count;
    sync writes with remaining budget;
return OK; // FIFO acceptance
```

```
Flush:
    poll STATUS until TX_IDLE;
return OK; // model: line idle
```

의사코드. 실제 RawUart_Write/Flush는 모든 단계 오류-deadline 검사. STATUS 반복 read 안전 조건.

하나의 작업 예산

S49 구조 예시

49_memory_budget.c

```
uint32_t left;
MemStatus st = Mem_Remaining(
    &io, start, timeout_ms, &left);
if (st != MEM_OK) return st;
st = Mem_Read32(&io, addr, &value, left);
if (st != MEM_OK) return st;
/* Check deadline after access too. */
```

start는 작업 전체에서 공유. timeout 1..INT32_MAX ms. callback은 받은 예산 내 반환.

시간이 실제로 흘러야 한다

IRQ를 막아 놓고 ISR tick을 기다리면 종료되지 않을 수 있다. CPU bus access 자체가 멈추면 C timeout 검사까지 돌아오지 못한다. 실제 primitive의 시간 상한도 필요하다.

Write / Flush

Write OK	모든 byte를 FIFO에 수락시킴. src는 반환 뒤 사용 안 함.
Flush OK	모델에서 shift register까지 idle 확인
n == 0	READY handle에서 src=NULL 허용, sent=0
동시성	한 소유자만 FIFO에 접근. IRQ·DMA와 혼용 금지.

sent가 정확한 retry 위치는 아니다

성공 반환한 write만 sent에 센다. 실패한 write도 장치에 도달했을 수 있다. 따라서 sent부터 무조건 재전송하면 같은 byte를 두 번 보낼 수 있다.

실패 결과를 읽는 법

write 실패	해당 byte는 sent 미포함. 실제 도달 여부 불확실.
sync 실패	직전 성공 write는 sent 포함. 전달 완료 불확실.
TX timeout	이 모델은 FAULT. 이미 들어간 byte는 계속 나갈 수 있음.
재사용	데이터시트의 recovery 후 재초기화. 자동 retry 없음.

fake에서 확인할 것

- Init write 순서와 각 sync 위치.
- TX_READY가 0이면 DATA write하지 않는가?
- TX_READY와 TX_IDLE을 다른 조건으로 보는가?
- 한 명령 실패·tick wrap·부분 전송 상태를 검사.
- write를 반영한 뒤 실패를 반환하는 경우도 시험.

포팅·배리어·첫 실물 시험

07 / BRING-UP · 필요한 책임을 하나씩 구현하고 근거를 남긴다.

실제 command 함수 연결

S50 구조 예시

50_port_binding.c

```
MemIo io = {
    .ctx = command_context,
    .read32 = Platform_Read32,
    .write32 = Platform_Write32,
    .sync = Platform_SyncWrites,
    .now_ms = Platform_MonotonicMs
};
```

Platform_*는 예시 이름이며 실제 API가 아님. port_binding.example.c의 원형에 맞춰 구현.

배리어를 넣기 전에 구분

volatile	컴파일러의 장치 접근 취급. lock 아님.
memory attribute	장치 영역의 cache-speculation 등 속성
DMB / DSB	Arm의 순서 / 더 강한 완료 조건. ISA 계약 확인.
ISB	명령 실행 문맥 동기화. 범용 I/O flush 아님.
DMA cache	CPU cache와 DMA RAM 일관성. barrier만으로 대체 안 됨.
sync	플랫폼 쓰기 전달·순서. 장치 기능 완료는 상태로 확인.

sync를 no-op으로 뒤도 되는가

접근 backend가 필요한 순서·완료를 이미 보장할 때만 가능하다. 지연 반영되는 쓰기(posted write)를 확인할 때도 문서상 안전한 register만 읽는다. RC/FIFO를 임의로 읽어 flush하지 않는다.

첫날 구현 순서

1. 계약	주소 공간·폭·endian·명령 오류·timeout 확인
2. 보드	전원 → clock/reset/pinmux, 실제 문서 순서
3. 읽기	안전한 ID/version/status를 한 번 읽기
4. 쓰기	정책 확인한 설정 하나를 적용·안전한 확인
5. 데이터	UART 1 byte, 상대 수신기·파형으로 확인
6. 실패	timeout·부분 전송·복구 경로 확인
7. 확장	polling → RX → IRQ → DMA, 필요할 때만

다른 장치에 재사용할 패턴

- 주소·접근 계약과 register 의미를 분리한다.
- config 검증 → encode → 적용 → 상태 공개.
- 단일 소유자·전체 timeout·명시적 오류 계약.
- RO/RW/W1C/RC/FIFO별 전용 동작.
- GPIO·timer·SPI·I2C·DMA는 실제 순서를 각각 구현.

controller와 외부 센서 주소

센서의 register offset을 CPU base에 더해서 접근하지 않는다. I2C/SPI controller의 register를 조작해 버스 거래를 만들어야 한다. 영상 HAL_I2C_Mem_Read가 하던 START·주소·데이터·오류·STOP을 직접 맡는 것이다.

제공 범위

새 템플릿은 polling TX 중심 모델이다. 실제 map·board bring-up·RX·IRQ·DMA는 미구현이다. 시험 상태는 memory_io/README와 quality-check에 구분해서 기록한다.

핸들·설정·출력 계약

08 / API DESIGN · 입력 설정과 동작 중인 객체를 구분한다.

공통 결과 타입

S01 C17

01_status.h

```
#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

typedef enum {
    DRV_OK = 0,
    DRV_EARG,
    DRV_ESTATE,
    DRV_EBUSY,
    DRV_ETIMEOUT,
    DRV_EIO,
    DRV_EID
} DrvStatus;
```

공통 타입. 각 드라이버의 정책에 맞춰 확장.

API의 기본 모양

S02 구조 예시

02_api_shape.h

```
/* Opaque types: define in your module. */
typedef struct Device Device;
typedef struct Config Config;
typedef struct Sample Sample;

DrvStatus device_init(
    Device *dev, const Config *cfg);
DrvStatus device_read(
    Device *dev, Sample *out);
```

S01. 불투명 타입의 정의·생성 정책은 별도.

각 포인터의 목적

Device *dev	내가 관리하는 장치 객체. 상태를 갱신할 수 있다.
const Config *cfg	읽어서 적용할 설정. 이 경로로는 수정하지 않는다.
Sample *out	성공 결과를 저장할 호출자 공간
const uint8_t *src	읽기 전용 입력 바이트
uint8_t *dst + n	수정 가능한 버퍼와 허용 길이

& / * / . / ->

S03 C17

03_handle_demo.c

```
typedef struct { int ready; } Demo;

int handle_demo(void)
{
    Demo d = {0};
    Demo *p = &d;
    p->ready = 1;
    return d.ready; /* returns 1 */
}
/* p->ready == (*p).ready */
```

독립 함수 예. handle_demo()는 1을 반환.

handle에 넣을 것 / 빌릴 것

- 장치별: 주소, 실제 적용 설정, scale, 유효성, 상태.
- 빌리는 객체: HAL handle, bus context, ops 테이블.
- 빌린 객체는 사용 기간 전체에 걸쳐 살아 있어야 한다.
- struct 복사는 포인터 값도 복사한다. 장치가 복제되지는 않는다.
- 장치별 상태를 함수 내부 static 하나에 모으지 않는다.

API 선언 옆에 적을 6가지

단위	길이는 bytes? elements? 시간은 ms?
실행	동기 완료인가, 비동기 시작인가?
수명	함수 반환 후 버퍼·context를 보관하는가?
출력	실패하면 out을 유지하는가? 일부 변경 가능한가?
문맥	ISR 허용? task 전용? 누가 잠그는가?
재호출	busy·재초기화·취소 후 호출 규칙은?

숫자 0으로 오류를 표현하지 않는다

0°C·0g·레지스터 값 0은 정상일 수 있다. 상태는 반환값으로, 데이터는 out 포인터로 전달한다. 성공을 확인한 뒤 out을 사용한다.

함수 포인터 + context

09 / DEPENDENCY INJECTION · 무엇을 읽을지와 어떻게 읽을지를 나눈다.

함수 타입과 함수 포인터

S04 C17

04_reg_bus.h

```
typedef DrvStatus (*ReadReg)(
    void *ctx, uint8_t reg,
    uint8_t *dst, size_t n,
    uint32_t timeout_ms);

typedef DrvStatus (*WriteReg)(
    void *ctx, uint8_t reg,
    const uint8_t *src, size_t n,
    uint32_t timeout_ms);

typedef struct {
    ReadReg read;
    WriteReg write;
} BusOps;

typedef struct {
    const BusOps *ops;
    void *ctx;
} RegBus;
```

S01. 동기 전체 전송, 주소 폭 8비트인 인터페이스.

호출 순서

S05 구조 예시

05_bus_call.c

```
uint8_t id = 0;
DrvStatus st = bus.ops->read(
    bus.ctx, reg_id, &id, 1U, 20U);

if (st == DRV_OK) {
    /* Check id before using device. */
}
```

S04. bus-reg_id는 유효하게 설정.

이 인터페이스의 약속

- DRV_OK = 요청한 전체 바이트 전송 완료.
- 반환 뒤 src/dst를 보관하지 않는 동기 callback.
- 실패한 dst는 일부 변경되어 있을 수 있다.
- reg-주소 폭·최대 n-timeout 단위는 어댑터 계약.

context로 장치별 연결을 전달

S06 C17

06_fake_read.h

```
typedef struct {
    uint8_t bytes[256];
} Fake;

static DrvStatus fake_read(
    void *ctx, uint8_t reg,
    uint8_t *dst, size_t n,
    uint32_t timeout_ms)
{
    Fake *f = ctx;
    (void)timeout_ms;
    if (!f || !dst || n == 0U ||
        n > sizeof f->bytes - reg) {
        return DRV_EARG;
    }
    for (size_t i = 0; i < n; ++i) {
        dst[i] = f->bytes[reg + i];
    }
    return DRV_OK;
}
```

S01. 읽기 전용 메모리 모델이며 실제 레지스터 부작용 모델은 아님.

같은 코드를 다른 context와 사용

S07 구조 예시

07_fake_bind.c

```
static const BusOps fake_ops = {
    .read = fake_read, .write = NULL
};

Fake f = {0};
RegBus bus = { &fake_ops, &f };
/* Use only read; write is unsupported. */
```

S04 + S06. f의 수명 안에서만 bus 사용.

함수와 객체의 타입을 맞추다

- ops는 공유 가능하지만 ctx는 해당 함수가 기대하는 객체여야 한다.
- HAL 함수 시그니처가 다르면 wrapper를 만든다. 함수 포인터 cast로 맞추지 않는다.
- C: Fake *f = ctx; C++: static_cast<Fake*>(ctx).
- write=NULL처럼 미지원 연산은 호출 전에 검사한다.

데이터시트 → 구현 계획

10 / REGISTER CONTRACT · 주소보다 접근 조건과 의미를 먼저 적는다.

작업 시작 전에 채울 표

부품 / 판	정확한 part number, datasheet rev, errata
회로	전원, 인터페이스 선택, 주소 strap, IRQ 핀
버스	7비트 주소, SPI mode, clock, dummy, CS
식별	ID 주소·mask·기대값
시간	전원 안정화, reset 완료, 첫 변환 대기
설정	range·ODR·filter·단위·금지 조합
읽기	burst 길이, endian, 유효 비트, 부호, latch
실패	timeout·부분 적용·재시도 가능 조건

레지스터별 최소 메모

S08 구조 예시

08_register_worksheet.txt

```
/* Register worksheet - NOT real map. */
/* name: CTRL */
/* address: fill from datasheet */
/* access: RW / RO / WO / W1C / ... */
/* reset: value and valid condition */
/* fields: mask, shift, encoding */
/* reserved: preserve/write-0/... */
/* precondition: standby/ready/... */
/* readback: stable mask only */
```

실제 부품의 데이터시트로 각 항목을 채운다.

세 종류의 주소

addr7	I2C 버스에서 어느 장치를 선택하는가
reg	그 장치 내부의 어느 레지스터인가
pointer	MCU RAM 또는 MMIO의 어느 객체인가

초기화는 순서가 있는 절차

연결 검사 → 전원/ready → ID → 필요한 reset → 설정 가능한 상태 → 설정 적용 → 의미 있는 검증 → 측정 시작 → 첫 샘플 조건 → READY.

- 이 순서는 틀이다. ID/reset 순서도 부품에 따라 다르다.
- reset/start 같은 명령은 반복해도 같은 의미인지 확인한다.
- 실패하면 READY로 표시하지 않는다. 부분 적용 가능성을 남긴다.

설정 갱신의 순서

S09 구조 예시

09_config_sequence.txt

```
/* Control-flow recipe, not an API. */
validate_requested_config();
encode_register_values();
enter_required_device_state();
write_configuration();
verify_stable_fields_if_supported();
resume_and_wait_if_required();
commit_applied_config_and_scale();
```

각 단계의 실패를 검사. 실제 함수가 아닌 설계 순서.

requested와 applied를 구분

- 쓰기 전에 scale을 바꾸지 않는다.
- 일부 쓰기 뒤 실패하면 이전 scale도 확신할 수 없다. config_valid=false로 두고 복구한다.
- reset 후 bank-FIFO·샘플·설정 cache의 유효성을 다시 판단한다.
- readback은 안정된 RW 필드만 mask 비교한다. WO-self-clearing에는 다른 완료 조건을 쓴다.

무엇을 읽을지 모를 때

센서 의미	센서 datasheet + register map
MCU 동작 순서	MCU reference manual + errata
함수 인수·상태	해당 SDK의 header + source
코어·IRQ	Arm/CMSIS 문서

비트 필드 접근 정책

11 / BITS · 마스크 계산과 하드웨어 쓰기의 의미를 분리한다.

일반 RW 필드의 순수 값 계산

S10 C17

10_fields.h

```
static inline uint8_t field_get8(
    uint8_t reg, uint8_t mask,
    unsigned shift)
{
    return (uint8_t)(
        ((uint32_t)reg & mask) >> shift);
}

static inline uint8_t field_put8(
    uint8_t old, uint8_t mask,
    unsigned shift, uint8_t value)
{
    uint32_t keep =
        (uint32_t)old & ~(uint32_t)mask;
    uint32_t put =
        ((uint32_t)value << shift) & mask;
    return (uint8_t)(keep | put);
}
```

stdint.h. shift<8, mask/encoding은 유효. 값 범위는 호출 전에 검증.

비트 3:1에 mode=5 넣기

S11 구조 예시

11_field_use.c

```
const uint8_t mask = 0x0EU;
uint8_t next = field_put8(
    old, mask, 1U, 5U);
uint8_t mode = field_get8(
    next, mask, 1U);
/* mode == 5; other bits preserved. */
```

S10. old는 이미 읽은 일반 RW 값. 가상 필드 예.

연산 읽기

$x \& \text{mask}$	선택 비트만 남김
$x \text{mask}$	선택 비트를 1로 만듦
$x \& \sim \text{mask}$	선택 비트를 0으로 만듦
$x \wedge \text{mask}$	선택 비트 반전
$\ll / \gg$	명시한 폭의 unsigned 값에서 이동

읽기·쓰기 의미별 선택

RO	읽기만. status 읽기에 부작용이 있는지도 확인.
RW	조건이 맞으면 read-modify-write 가능.
WO	기존 값을 읽어 복원하려 하지 않는다.
W1C	지울 비트만 1로 써서 acknowledge.
W0C	지울 비트가 0. 나머지·예약 비트의 규칙도 확인.
W1S	설정할 비트만 1로 쓴다.
RC / FIFO	읽는 행위가 지우기·꺼내기일 수 있다.
self-clearing	쓴 값 유지 대신 완료 조건으로 검증.

W1C는 RMW를 사용하지 않는다

S12 구조 예시

12_w1c_write.c

```
/* Fictional register: all status bits */
/* are W1C, unused bits accept zero. */
uint8_t clear_mask = 0x04U;
st = bus.ops->write(
    bus.ctx, status_reg,
    &clear_mask, 1U, 20U);
```

S04. 실제 register별 W1C·예약 비트 규칙 확인.

RMW를 쓸 수 있는 조건

- 읽은 값이 유효하고 읽기에 파괴적 부작용이 없다.
- 읽기와 쓰기 사이 동시 변경자를 통제한다.
- 예약 비트·W1C·하드웨어 갱신 비트를 함께 재기록하지 않는다.
- bus lock은 거래를 보호하고 device lock은 여러 거래의 설정 절차를 보호할 수 있다.

bitfield는 wire format이 아니다

C bitfield의 배치·padding·접근 폭을 임의로 레지스터 맵과 같다고 가정하지 않는다. 통신 버퍼·MMIO에는 mask/shift와 장치별 접근 함수를 우선 사용한다.

I2C·SPI 어댑터

12 / HAL COMPARISON · STM32 비교 학습용. 현재 업무 구현은 2~7쪽 우선.

STM32F4 HAL I2C 읽기 어댑터

S13 HAL

13_i2c_port.h

```
typedef struct {
    I2C_HandleTypeDef *hal;
    uint8_t addr7;
} I2cPort;

static DrvStatus i2c_read(
    void *ctx, uint8_t reg,
    uint8_t *dst, size_t n,
    uint32_t timeout_ms)
{
    I2cPort *p = ctx;
    if (!p || !p->hal || !dst ||
        !n || n > UINT16_MAX ||
        p->addr7 > 0x7FU) {
        return DRV_EARG;
    }
    HAL_StatusTypeDef s = HAL_I2C_Mem_Read(
        p->hal, (uint16_t)(p->addr7 << 1),
        reg, I2C_MEMADD_SIZE_8BIT,
        dst, (uint16_t)n, timeout_ms);
    switch (s) {
        case HAL_OK: return DRV_OK;
        case HAL_BUSY: return DRV_EBUSY;
        case HAL_TIMEOUT: return DRV_ETIMEOUT;
        default: return DRV_EIO;
    }
}
```

S01 + stm32f4xx_hal.h. 초기화된 버스, 8비트 reg, 동기 읽기.

복사 전에 확인

- 이 주소 shift는 기존 F4 HAL의 인수 계약이다.
- HAL 오류를 전부 NACK이라고 해석하지 않는다. 상세 원인은 HAL error code로 추적한다.
- n은 캐스팅 전에 검사한다. 긴 거래를 나누려면 프로토콜도 분할을 허용해야 한다.
- 공유 버스 소유권과 timeout 문맥은 호출자가 보장한다.

I2C register read의 파형

START → Addr+W → Register → repeated START → Addr+R → Data → 마지막 NACK → STOP. 각 바이트의 ACK/NACK도 포함한다.

- Transmit(reg) 뒤 Receive() 두 호출이 같은 파형을 보장하지는 않는다.
- MemAddSize는 내부 주소 길이. 센서 데이터의 분해능이 아니다.
- addr7=0x1D와 reg=0x08, RAM buffer 주소는 서로 다르다.

ADXL355 9바이트 읽기

S14 HAL

14_adxl355_read.c

```
uint8_t raw[9];
HAL_StatusTypeDef st = HAL_I2C_Mem_Read(
    &hi2c1, (uint16_t)(0x1DU << 1),
    0x08U, I2C_MEMADD_SIZE_8BIT,
    raw, (uint16_t)sizeof raw, 20U);
if (st == HAL_OK) {
    /* Decode only complete read. */
}
```

버스·센서가 준비된 후 호출. 주소는 ASEL 회로와 일치해야 함.

SPI 프레임 체크

전기 / 클록	CPOL-CPHA-bit order-최대 SCK
명령	read/write bit, 주소 폭, burst bit
지연	dummy byte/clock과 유효 RX 시작 위치
CS	command부터 payload 끝까지 유지 조건
버스 공유	장치별 mode-속도 전환과 lock 범위

시간 예산

9바이트 데이터 + 장치 주소 2회 + reg 1바이트 = 12×9 = 108클록. 400 kHz에서 약 270 μs, 100 kHz에서 약 1.08 ms. START/STOP-stretching·소프트웨어 지연은 추가된다.

HAL·LL 선택과 초기화

13 / HAL COMPARISON · STM32 비교 학습용. 현재 업무 구현은 2~7쪽 우선.

HAL과 LL의 차이

HAL	기능·전송 절차 제공. 통신 handle의 상태·버퍼·길이를 관리.
LL	플래그·레지스터·개별 기능 조작. 순서와 상태는 사용자 코드가 구성.
polling / IT / DMA	HAL/LL과 별개인 진행 방식. 둘 다 선택 가능.
CMSIS	코어 기능과 장치의 register layout·주소·비트 정의.

GPIO 하나로 비교

S15 HAL

15_gpio_compare.c

```
/* Alternatives; PA5 output is ready. */
HAL_GPIO_WritePin(
    GPIOA, GPIO_PIN_5, GPIO_PIN_SET);

LL_GPIO_SetOutputPin(GPIOA, LL_GPIO_PIN_5);

GPIOA->BSRR = (1UL << 5);
```

HAL + stm32f4xx_ll_gpio.h. 클럭·출력 설정 후 한 방법 선택.

자주 혼동하는 경계

- 영상의 low-level 함수는 HAL wrapper다. ST LL 사용을 뜻하지 않는다.
- 기존 F4 HAL이 반드시 LL을 호출하는 것은 아니다. 모듈·HAL 세대를 구분한다.
- LL ReceiveData8은 MCU DR을 읽는다. 센서에 새 거래를 요청하거나 기다리지 않는다.
- LL은 bit-banging이 아니다. 신호는 주변장치 하드웨어가 생성한다.

어디를 읽으면 준비 상태를 알까?

HAL_Init	공통 기반·time base. 모든 주변장치 init 이 아님.
SystemClock_Config	시스템·버스·주변장치 클럭
MX_*_Init	프로젝트 생성 코드의 핀·주변장치 설정
HAL_*_MspInit	보드별 클럭·핀·DMA·NVIC 연결
*_it.c	실제 IRQ → HAL handler 또는 사용자 ISR
sensor_init	외부 부품의 ID·동작 레지스터·안정화

RAM handle / 하드웨어 instance

S16 HAL

16_hal_handle.c

```
static I2C_HandleTypeDef hi2c1 = {0};
/* Part of initialization only: */
hi2c1.Instance = I2C1;

/* &hi2c1: RAM management object */
/* I2C1: peripheral registers */
```

설명 조각. 이것만으로 I2C 초기화가 끝나지 않음.

혼용과 성능의 기준

- 한 instance의 진행 중 전송은 한 구현이 책임진다.
- I2C1=HAL, TIM2=LL처럼 역할을 나누면 추적하기 쉽다.
- HAL 전송 도중 LL로 flag·ACK·STOP·DMA를 바꾸면 상태가 어긋날 수 있다.
- _HAL_LOCK은 기존 F4에서 RTOS mutex가 아니다.
- LL의 작은 연산/inline이 전체 거래의 원자성을 보장하지 않는다.
- API 비용·CPU 점유·버스 시간·최악 지연을 나눠 측정한다.

UART 초기화 템플릿

14 / HAL COMPARISON · STM32 비교 학습용. 현재 업무 구현은 2~7쪽 우선.

타입 있는 설정과 핸들

S17 HAL

17_uart_api.h

```
typedef struct {
    USART_TypeDef *instance;
    uint32_t baud_rate;
} UART_Config;

HAL_StatusTypeDef UART_Init(
    UART_HandleTypeDef *handle,
    const UART_Config *config);
```

전체 파일은 existing/uart_template.h. 8N1-TX/RX 고정.

UART_Init 본체

S18 HAL

18_uart_init.c

```
HAL_StatusTypeDef UART_Init(
    UART_HandleTypeDef *h,
    const UART_Config *c)
{
    if (!h || !c) return HAL_ERROR;
    if (!IS_UART_INSTANCE(c->instance) ||
        !c->baud_rate ||
        !IS_UART_BAUDRATE(c->baud_rate)) {
        return HAL_ERROR;
    }
    if (h->gState != HAL_UART_STATE_RESET ||
        h->RxState != HAL_UART_STATE_RESET) {
        return HAL_BUSY;
    }
    h->Instance = c->instance;
    h->Init.BaudRate = c->baud_rate;
    h->Init.WordLength = UART_WORDLENGTH_8B;
    h->Init.StopBits = UART_STOPBITS_1;
    h->Init.Parity = UART_PARITY_NONE;
    h->Init.Mode = UART_MODE_TX_RX;
    h->Init.HwFlowCtl = UART_HWCONTROL_NONE;
    h->Init.OverSampling = UART_OVERSAMPLING_16;
    return HAL_UART_Init(h);
}
```

S17. 기존 전체 템플릿과 중복 링크하지 말 것.

호출부: 객체를 만들고 주소 전달

S19 HAL

19_uart_start.c

```
static UART_HandleTypeDef uart = {0};

HAL_StatusTypeDef start_uart(void)
{
    const UART_Config cfg = {
        .instance = USART2,
        .baud_rate = 115200U
    };
    return UART_Init(&uart, &cfg);
}
```

S17+S18 또는 existing 템플릿. 최초 초기화용.

메모리와 설정의 관계

uart	프로그램이 유지하는 실제 관리 객체
&uart	UART_Init이 수정할 객체의 주소
cfg / &cfg	원하는 설정과 그 주소
h->Init	호출자가 만든 handle의 멤버
HAL_UART_Init	ST가 제공한 하드웨어 초기화 함수

프로젝트 연결 전제

- HAL_Init와 시스템 클럭을 먼저 준비한다.
- MSP에서 실제 USART 클럭-TX/RX alternate-function 핀을 설정한다.
- 바운드 검사만으로 baud 오차·전기 조건이 검증되지는 않는다.
- 이 템플릿은 RESET handle만 init. 재설정은 안전한 DeInit 후 한다.
- 기존 CubeMX huart2가 있으면 같은 USART2의 handle을 추가 생성하지 않는다.
- 설정 구조체는 값만 복사한다. handle은 이후 사용 중 살아 있어야 한다.

UART 송신·가변 인자

15 / HAL COMPARISON · STM32 비교 학습용. 현재 업무 구현은 2~7쪽 우선.

■ 바이트 단위 동기 송신

S20 HAL

20_uart_write.c

```
HAL_StatusTypeDef UART_Write(
    UART_HandleTypeDef *h,
    const uint8_t *data,
    uint16_t n, uint32_t timeout_ms)
{
    if (!h || !data || !n) return HAL_ERROR;
    if (h->Init.WordLength !=
        UART_WORDLENGTH_8B ||
        h->Init.Parity != UART_PARITY_NONE ||
        h->Init.StopBits != UART_STOPBITS_1 ||
        !(h->Init.Mode & UART_MODE_TX)) {
        return HAL_ERROR;
    }
    return HAL_UART_Transmit(
        h, data, n, timeout_ms);
}
```

준비된 8N1 handle. task/main 전용, 접근은 외부 직렬화.

■ 문자열은 끝의 NUL을 제외

S21 HAL

21_uart_send.c

```
const uint8_t msg[] = "Hello\r\n";
HAL_StatusTypeDef st = UART_Write(
    &uart, msg,
    (uint16_t)(sizeof msg - 1U), 100U);
if (st != HAL_OK) {
    /* Some bytes may already be sent. */
}
```

S20 선언 + 초기화된 uart. 실패 시 전송 rollback은 없음.

■ 길이의 단위를 고정

- 이 함수의 n은 1~65535 bytes.
- F4 HAL 9비트/no-parity는 16비트 원소를 읽으므로 여기서 거절한다.
- sizeof(pointer)는 버퍼 길이가 아니다.
- IT/DMA로 바꾸면 지역 버퍼 수명과 완료 계약을 다시 설정한다.

■ 선언의 ... = 가변 인자

S22 구조 예시

22_ellipsis.txt

```
void f(int a, ...); /* declaration */
/* Calls supply actual values: */
f(10);
f(10, 20, 30);
```

인자 개수 관점의 문법 예. f의 실제 계약·구현은 별도.

■ count개 int를 읽는 계약

S23 C17

23_varargs.c

```
#include <stdarg.h>
#include <stdio.h>

void PrintInts(int count, ...)
{
    va_list ap;
    va_start(ap, count);
    for (int i = 0; i < count; ++i) {
        int v = va_arg(ap, int);
        printf("%d\n", v);
    }
    va_end(ap);
}
/* PrintInts(3, 10, 20, 30); */
```

count>=0, 실제 추가 인자는 count개의 int. stdout 연결 별도.

■ va_arg는 타입 변환기가 아니다

- 선표는 구분자. 호출 시 f(10, ...)라고 쓰지 않는다.
- 첫 인자가 자동으로 개수가 되지 않는다. count/format 규칙을 직접 정한다.
- float는 double로 승격: va_arg(ap, double).
- STM32의 작은 정수는 보통 int로 승격. 일반 규칙은 int 또는 unsigned int.
- 개수·승격 타입이 맞지 않으면 정의되지 않은 동작이 될 수 있다.
- 설명에서 생략 표시로 쓴 ...와 실제 C 선언 문법을 구분한다.

바이트 → raw → 물리량

16 / DECODE · 전송·해독·변환·공개를 각각 확인한다.

명시적인 endian 해독

S24 C17

24_endian.h

```
static inline uint16_t u16_be(
    const uint8_t b[2])
{
    return (uint16_t)(
        ((uint32_t)b[0] << 8) | b[1]);
}

static inline uint16_t u16_le(
    const uint8_t b[2])
{
    return (uint16_t)(
        b[0] | ((uint32_t)b[1] << 8));
}
```

stdint.h. b는 읽을 수 있는 2바이트.

상위 정렬된 signed 20비트

S25 C17

25_signed20.h

```
static inline int32_t s20_left(
    const uint8_t b[3])
{
    uint32_t u = ((uint32_t)b[0] << 12)
        | ((uint32_t)b[1] << 4)
        | ((uint32_t)b[2] >> 4);
    if (u & UINT32_C(0x80000)) {
        return (int32_t)u -
            INT32_C(1048576);
    }
    return (int32_t)u;
}
```

stdint.h. b는 3바이트, 하위 nibble은 측정값이 아닌 형식.

빠른 경계값

00 00 00	0
00 00 10	1
7F FF F0	524287
80 00 00	-524288
FF FF F0	-1

ADXL355: 명목 스케일

S26 C17

26_adxl355_units.h

```
static inline uint16_t temp12(
    const uint8_t b[2])
{
    return (uint16_t)(
        (((uint32_t)b[0] & 0x0FU) << 8)
        | b[1]);
}

static inline float temp_c(uint16_t raw)
{
    return 25.0f +
        ((float)raw - 1885.0f) / -9.05f;
}

static inline float accel_2g(int32_t raw)
{
    return ((float)raw / 256000.0f)
        * 9.80665f;
}
```

stdint.h. 현행 기준 ±2g 설정일 때. 개별 calibration·범위 변경은 별도.

성공한 완성 샘플만 공개

S27 구조 예시

27_commit_sample.c

```
/* Control flow; provide concrete APIs. */
Sample next;
st = read_bytes(dev, bytes, sizeof bytes);
if (st != DRV_OK) return st;
st = decode_sample(dev, bytes, &next);
if (st != DRV_OK) return st;
*out = next;
return DRV_OK;
```

dev/out 검증 후. decode가 next 전체를 초기화한다는 계약.

값이 맞는지 확인할 때

- LSB/unit이면 나누고 unit/LSB이면 곱한다.
- 실제 적용된 range와 scale을 함께 유지한다.
- burst가 같은 시점 샘플을 자동 보장하지 않는다. latch-DRDY-sequence-FIFO 조건 확인.
- struct 대입은 논리적 공개다. ISR/task 간 원자적인 공개는 별도 동기화한다.

시간 제한·오류·복구

17 / FAIL PREDICTABLY · 실패 원인을 보존하고 다음 호출 조건을 정한다.

unsigned tick 차이로 경과시간

S28 C17

28_timeout.h

```
static inline bool expired(
    uint32_t now, uint32_t start,
    uint32_t budget)
{
    return (uint32_t)(now - start)
        >= budget;
}

static inline uint32_t time_left(
    uint32_t now, uint32_t start,
    uint32_t budget)
{
    uint32_t used = now - start;
    return used >= budget ? 0U
        : budget - used;
}
```

stdint.h + stdbool.h. 동일 단위·단조 tick, 한 전체 wrap 이전에 관측.

전체 작업 budget을 전달

S29 구조 예시

29_operation_budget.c

```
uint32_t start = tick_ms();
uint32_t left = time_left(
    tick_ms(), start, operation_ms);
if (!left) return DRV_ETIMEOUT;
st = step_one(left);
if (st != DRV_OK) return st;
left = time_left(
    tick_ms(), start, operation_ms);
if (!left) return DRV_ETIMEOUT;
return step_two(left);
```

S28. 각 step의 timeout 계약이 전체 budget에 부합해야 함.

시간이 실제로 흐르는가?

- tick이 IRQ에 의존하면 IRQ 금지·우선순위를 확인한다.
- sleep 중 tick 중단·보정 조건을 확인한다.
- 전송 timeout과 startup/전체 init timeout은 다르다.
- 각 호출에 100 ms를 주면 여러 호출 전체가 100 ms인 것은 아니다.

상태에 따라 대응을 나눈다

EARG	설정·코드 계약 수정. 반복 전송하지 않음.
ESTATE	초기화·샘플 준비·유효성 확인.
EBUSY	소유자·진행 중 요청·큐 정책 확인.
ETIMEOUT	시간 근거·장치 준비·버스 정체 확인.
EIO	원래 오류 코드·단계·레지스터를 함께 기록.
EID	실제 부품·주소·전원·인터페이스 확인.

재시도 전에 물어볼 질문

- 앞선 쓰기가 실제 적용되었을 가능성은?
- 한 번 더 실행해도 같은 의미인가?
- FIFO pop·샘플 trigger·reset은 중복 실행이 안전한가?
- 최초 오류와 복구 오류를 따로 보존하는가?
- 재시도 횟수·backoff·재초기화 조건이 정해져 있는가?

설정·bank·cache의 유효성

reset / 전원 재시작	applied config·bank·샘플 valid 상태 재판단
bank 선택 + 접근	하나의 동기화 범위로 보호
status / FIFO	일반 설정 cache처럼 재사용하지 않음
부분 설정 실패	이전 scale을 망신하지 않고 재설정 필요 상태로
readback	명령 bit보다 안정된 mask·완료 조건 확인

복구 책임의 경계

버스 계층은 거래를 정리하고, 장치 계층은 설정·샘플 유효성을 정리한다. 응용은 재시도·격리·사용자 알림 정책을 정한다. 낮은 계층에서 무한 재시도하지 않는다.

ISR·DMA·RTOS 수명

18 / ASYNC · 시작 성공과 완료 성공을 분리한다.

최소 알람: coalescing flag

S30 C17

30_pending_flag.c

```
#include <stdatomic.h>
#include <stdbool.h>

_Static_assert(ATOMIC_BOOL_LOCK_FREE == 2,
    "Need always lock-free atomic bool");
static atomic_bool pending =
    ATOMIC_VAR_INIT(false);

void signal_from_isr(void)
{
    atomic_store_explicit(
        &pending, true, memory_order_relaxed);
}

bool take_pending(void)
{
    return atomic_exchange_explicit(
        &pending, false, memory_order_relaxed);
}
```

대상 compiler/ABI가 ISR atomic 사용을 허용해야 함. payload 없는 알람.

이 flag의 의미

- 여러 이벤트를 하나로 합친다. 이벤트 개수·샘플 수를 보존하지 않는다.
- payload 공개용 동기화가 아니다. 데이터 전달에는 큐·소유권 프로토콜을 쓴다.
- 대상이 지원하지 않으면 RTOS의 ISR-safe 알람이나 적절한 임계구역을 사용한다.
- ISR에서는 짧게 알리고, 오래 걸리는 통신·변환은 task/main에서 한다.

소유권을 끊김 없이

submit → in-flight → 완료/오류/취소 처리 → 하드웨어가 더 이상 접근하지 않음 확인 → 버퍼 반환. 취소 요청만으로 버퍼가 안전해지지 않는다.

비동기 구현 점검표

handle / ctx	등록·진행 중 callback보다 긴 수명, 안정된 주소
TX buffer	완료 전 변경·해제·재사용 금지
RX buffer	완료·유효 길이 확인 후 소비
IRQ 경로	vector → 프로젝트 handler → 구현 handler → 통지
timeout / cancel	정확히 한 번 최종 결과 통지, 정리 후 재사용
FIFO	entry 크기·태그·watermark·overflow·유실 정책

네 가지 다른 보장

volatile	특별한 객체 접근 의미. mutex가 아님.
C atomic	CPU 관점 atomic-memory order. DMA cache 해결 아님.
bus lock	공유 컨트롤러의 거래 소유권.
device lock	여러 거래에 걸친 장치 설정·상태 보호.

DMA가 실제로 추가되면

- DMA가 접근 가능한 RAM 영역·정렬·전송 단위를 확인한다.
- cache가 있는 MCU는 방향에 맞는 clean/invalidate와 cache-line 소유권을 설계한다.
- 일반적인 cache 조작 코드를 모든 STM32에 붙이지 않는다. MCU별 지침을 따른다.
- ISR/task뿐 아니라 DMA·다른 코어·장치가 변경자인지 확인한다.
- 기본 F4 HAL lock을 task 간 mutex로 간주하지 않는다.

객체 종료 순서

새 요청 중지 → IRQ/콜백 등록·접근 종료 → 진행 중 전송 완료/취소 → 실행 중 콜백 종료 → 객체·버퍼 파괴. RAII도 이 비동기 순서를 자동으로 만들어 주지는 않는다.

C 포인터·버퍼 빠른 참조

19 / C17 CORE · 주소·크기·폭·수명을 명시한다.

포인터 선언을 읽는 순서

T *p	T 객체의 주소를 보관
const T *p	가리킨 값을 이 포인터로 수정하지 않음
T *const p	포인터가 가리키는 곳을 재지정하지 않음
const T *const p	두 제약을 함께 적용
T **out	호출자의 포인터 변수 자체를 바꿀 주소
T (*p)[N]	N개짜리 배열을 가리키는 포인터

배열 크기는 호출자에서 전달

S31 구조 예시

31_array_size.c

```
uint16_t values[6] = {0};
size_t count = sizeof values /
                sizeof values[0];
size_t bytes = sizeof values;
/* count=6; bytes=6*sizeof(uint16_t) */

void parse(const uint8_t *p, size_t n);
/* Parameter uint8_t p[6] also adjusts */
/* to pointer; sizeof p is NOT six. */
```

stdint.h + stddef.h. 선언·호출 위치의 차이를 보는 조각.

덧셈 overflow 없는 범위 검사

S32 C17

32_bounds.h

```
static inline bool fits(
    size_t capacity, size_t offset,
    size_t count)
{
    return offset <= capacity &&
           count <= capacity - offset;
}
```

stddef.h + stdbool.h. 실제 객체도 capacity만큼 존재해야 함.

메모리 복사

memcpy	겹치지 않는 유효 영역 복사
memmove	겹칠 수 있는 유효 영역 복사
memset	바이트 채우기. int 값·C++ 생성자 대체 아님

정수 연산 전에 폭을 정한다

S33 구조 예시

33_integer_width.c

```
uint32_t word =
    ((uint32_t)hi << 8) | lo;

int64_t product = (int64_t)raw * scale;
/* Validate denominator and range */
/* before division/narrowing. */
```

stdint.h. raw*scale은 int64_t 안, hi/lo는 바이트라는 계약.

정수·부동소수점 체크

- uint8_t도 식에서 정수 승격될 수 있다.
- 왼쪽 이동 전 unsigned 폭을 잡고 shift<폭을 지킨다.
- signed overflow에 기대지 않는다. 축소 cast 전 표현 범위를 확인한다.
- float 상수는 필요하면 1.0f. 정수 나눗셈과 반올림 정책을 구분한다.
- 고정소수점은 중간 곱셈 폭·분모 0·최종 범위까지 검증한다.

포인터 cast로 프레임을 해독하지 않는다

(uint16_t)bytes는 정렬·별칭 규칙·CPU endian에 의존한다. wire format은 명시적 byte 조립으로 해석한다. packed도 이런 문제를 모두 해결하지 않는다.

const / volatile / restrict

const	이 경로에서 수정하지 않음. 다른 경로의 변경까지 막지는 않음.
volatile	주로 MMIO 접근. 동시성·순서 전체·DMA cache 보장 아님.
restrict	특정 접근의 비별칭 계약. 확신할 때만 사용.

수명 확인

반환한 지역 변수 주소, 끝난 config/context 주소, 진행 중 DMA의 지역 배열을 보관하지 않는다. static은 수명을 늘리지만 재진입·공유 문제를 따로 만든다.

C 구조체·링크지·제어 흐름

20 / C17 STRUCTURE · 모듈 경계와 데이터 표현을 분명하게 만든다.

태그를 가진 union

S34 C17

34_event.h

```
typedef enum {
    EV_COUNT, EV_TEMPERATURE
} EventKind;

typedef struct {
    EventKind kind;
    union {
        uint32_t count;
        float temperature_c;
    } data;
} Event;

static inline Event temperature_event(float c)
{
    return (Event){
        .kind = EV_TEMPERATURE,
        .data = { .temperature_c = c }
    };
}
/* Read only the member named by kind. */
```

stdint.h. C와 C++의 비활성 union 멤버 규칙을 혼동하지 말 것.

구조체는 저장 형식과 다를 수 있다

- struct에는 padding:정렬이 있다. sizeof 합계가 멤버 합과 같다고 가정하지 않는다.
- enum 크기는 C17에서 고정 폭이라고 가정하지 않는다.
- struct/enum 전체를 그대로 UART·파일 wire format으로 내보내지 않는다.
- memset으로 C++ 객체를 초기화하지 않는다.

switch로 유한 상태를 드러내기

S35 구조 예시

35_state_switch.c

```
switch (state) {
case READY:
    return perform_read();
case STARTING:
    return DRV_EBUSY;
default:
    return DRV_ESTATE;
}
```

S01. state·함수·상태 enum은 프로젝트에서 정의.

C/C++ 공용 헤더 틀

S36 C17

36_public_api.h

```
#ifndef DEVICE_API_H
#define DEVICE_API_H

#ifdef __cplusplus
extern "C" {
#endif

int device_reset(void);

#ifdef __cplusplus
}
#endif
#endif
```

C로 컴파일한 구현과 연결. C++ 클래스가 C로 변환되는 것은 아님.

이 키워드는 어디에 있는가?

static 함수	파일 밖에서 직접 연결되지 않는 도우미
static 지역 변수	호출 사이 값을 유지하는 공유 객체
extern T x	다른 곳의 객체를 선언. 정의는 한 곳.
static inline	작은 header 함수. 인라인 기계어 강제 아님.
f(void)	C17에서 인자가 없음을 명시
f()	C17 선언은 인자 정보 미지정. C++17과 구분

매크로보다 함수가 편한 경우

S37 C17

37_inline.h

```
static inline unsigned square(unsigned x)
{
    return x * x;
}
/* unsigned wrap is defined; choose */
/* input limits for your purpose. */
```

함수는 인자를 한 번 평가. SQUARE(i++) 같은 반복 평가 매크로와 구분.

오류 경로 정리

종속 단계는 실패 즉시 반환한다. 여러 자원의 해제가 필요하다면 획득 순서 반대로 정리하는 단일 cleanup 경로도 유용하다. 정리 실패가 최초 오류를 덮어쓰지 않도록 한다.

C++17 드라이버 문법

21 / C++ · 객체 수명에 자원 책임을 연결하되 비동기 종료를 따로 설계한다.

RAII: 잠금의 해제를 스코프에 연결

S38 C++17

38_guard.hpp

```
struct Mutex;
void lock(Mutex*) noexcept;
void unlock(Mutex*) noexcept;

class Guard {
public:
    explicit Guard(Mutex& m) noexcept
        : m_(m) { lock(&m_); }
    ~Guard() noexcept { unlock(&m_); }
    Guard(const Guard&) = delete;
    Guard& operator=(const Guard&) = delete;
    Guard(Guard&&) = delete;
    Guard& operator=(Guard&&) = delete;
private:
    Mutex& m_;
};
```

task 전용. lock은 획득 후 반환, unlock은 실패·예외 없음. 플랫폼 구현 필요.

반드시 읽을 표현

T& / const T&	객체 참조 / 이 경로로 수정하지 않는 참조
explicit	의도하지 않은 생성자 변환 제한
noexcept	예외를 밖으로 내보내지 않는 계약. 성공 보장 아님.
= delete	복사·이동 등 특정 연산 금지
[[nodiscard]]	반환값 무시 시 진단 유도
auto&	참조 유지. auto 값은 복사할 수 있음.
constexpr	상수 식 가능. 모든 호출의 컴파일 시 계산 강제 아님.

생성자 / init

생성자는 연결만 준비하고 실패할 수 있는 통신은 init()에서 상태 코드로 반환할 수 있다. 멤버 초기화 순서는 선언 순서다. class와 struct는 기본 접근 권한이 다르다.

C callback에서 C++ 객체로

S39 C++17

39_callback.cpp

```
class Reader {
public:
    void completed(int status) noexcept;
};

extern "C" void done(
    void* user, int status) noexcept
{
    if (!user) return;
    auto* r = static_cast<Reader*>(user);
    r->completed(status);
}
/* register_callback(&done, &reader); */
```

실제 callback typedef/ABI에 맞출 것. Reader 구현·등록 API 별도.

콜백과 이동의 수명

- Reader는 callback 등록·실행 동안 같은 주소에서 살아 있어야 한다.
- 비정적 멤버 함수 포인터는 일반 함수 포인터와 다르다.
- [this], [&local]은 원본 수명을 늘리지 않는다.
- std::move는 이동을 수행하라는 표식 성격. 소유권 이전 구현은 클래스 책임.

다형성과 컨테이너 선택

함수 포인터 + ctx	런타임 구현 교체, C ABI와 시험용 fake에 유용
virtual	런타임 다형성. 그 자체로 heap 필수 아님.
template<class Bus>	컴파일 시 구현 선택. 인라인 기회·코드 크기 확인.
std::array	고정 크기 값 배열, C++17 가능
span / bit_cast	C++20 기능. C++17에 그대로 붙이지 않음.
std::function	구현·capture에 따라 할당과 호출 비용 확인

완료 전 파괴하지 않는다

RAII가 자동으로 DMA·실행 중 callback을 취소하지 않는다. 명시적 stop/shutdown과 접근 종료를 보장한 뒤 자원을 파괴한다.

검증·찾아보기·원문 대응

22 / FINISH · 하드웨어에서만 확인할 조건과 순수 계산을 구분한다.

작은 성공의 증거를 남긴다

연결	전원·주소·핀 mux-pull-up-clock 확인
식별	ID 읽기와 기대값 비교
설정	유효 조합·필드 encoding-readback 가능 mask
해독	0:1·최대 양수·최소 음수·-1·reserved bit
오류	각 전송 단계에 실패 주입, out 유지 확인
실물	정지 자세·축 방향·온도 변화·파형 확인
확장	재초기화·busy·취소·FIFO overflow·DMA 수명

재사용 스니펫 찾기

S40~S50 / 2~7쪽	메모리 명령·주소·정책·내 UART·포팅
S01~S12 / 8~11쪽	handle-callback·데이터시트·bit field
S13~S23 / 12~15쪽	HAL 비교·UART 비교·가변 인자
S24~S29 / 16~17쪽	decode·단위·output-timeout
S30~S37 / 18~20쪽	ISR·버퍼·union·header
S38~S39 / 21쪽	C++ RAII·callback

검증 범위

기존 decode Python 산술 모델은 20비트 전체 패턴과 경계값을 통과했다. 새 memory_io의 C 테스트와 나머지 스니펫의 검토 범위는 각 README-quality-check.json을 확인한다. 실물 시험은 수행하지 않았다.

모든 기존 Markdown의 대응

07 메모리 구현	2~7쪽 주소·명령·내 UART·포팅
00 온보딩	1쪽 학습 경로, 22쪽 검증
01 영상 / 02 패턴	1·8~12·16쪽 사례·책임·계약
03 공통 패턴	2~7·10~12·16~18쪽
04 C/C++	8~9·11·15~21쪽
05 HAL/LL / 06 UART	12~15쪽 비교·가변 인자
목차 / 출처	1·22쪽 범위·출처
통합본	00~07 재수록, 중복 제외

주요 1차 자료

MMIO	Linux Device I/O, Arm CPU instruction reference
영상	youtube.com/watch?v=_JQAve05o_0
센서	Analog Devices ADXL354/ADXL355 Rev. D
HAL/LL	ST UM1725, stm32f4xx-hal-driver
MCU	ST RM0090, CMSIS device header
I2C	NXP UM10204
C	WG14 N1570 §§6, 7.16; SEI CERT
C++	C++17 draft, C++ Core Guidelines

복사해서 구현하는 순서

- S번호 → snippets/의 같은 번호 파일을 연다.
- 태그·needs·헤더·버퍼 길이·수명 계약을 읽는다.
- 현재 UART는 examples/memory_io/의 전체 파일 세트를 사용한다.
- 동일 함수 조각을 여러 번 링크하지 않는다.
- 실제 부품·명령 API에 맞춰 컴파일하고 경계값·실물 동작 확인.